

ARCHITECTURE & DATA MODEL SPECIFICATION / ENGINEERING
REFERENCE

SGR Feeder Platform

A multi-user transport coordination system connecting passengers arriving at or departing from the Mombasa SGR Terminus with shared and private cab services along the **North Coast** and **South Coast** corridors.

This document defines the production architecture, domain model, database schema, booking state machine, M-Pesa payment flows, location tracking strategy, and security model.

VERSION	1.0
DATE	August 2026
AUDIENCE	Engineering Team
STATUS	Approved for Implementation

Table of Contents

1. Executive Summary4

1.1 Platform Scope4

1.2 Architecture Principles4

1.3 Document Status5

2. System Architecture Overview6

2.1 The Trust Boundary6

2.2 Why Modular Monolith6

2.3 The Offline Edge7

3. Technology Stack Decisions8

3.1 What This Stack Is Not9

4. Domain Model10

4.1 Booking vs Ride vs Trip10

4.2 Entity Inventory10

5. Database Schema (Full SQL DDL)12

5.1 Extensions and Conventions12

5.2 Identity & Profiles12

5.3 Geography (Stations, Service Areas, Pickup Points)14

5.4 Operations (Journeys, Bookings, Rides, Stops)15

6. Booking State Machine17

6.1 Transition Rules18

6.2 Concurrency: Holding Seats Without Selling Them Twice19

7. Payment Flows20

7.1 Payment Lifecycle20

7.2 Idempotency20

7.3 Payment Schema (DDL)21

7.4 Provider Abstraction21

8. Location Tracking Strategy	23
8.1 Two Concepts: Current Position vs Historical Events	23
8.2 Adaptive GPS Frequency	23
8.3 Android Background Location Reality	24
9. Security & Authentication	25
9.1 Authentication	25
9.2 Role Hierarchy	25
9.3 Authorization Matrix	25
9.4 Driver Verification	26
9.5 Audit Logging	26
10. API Contract	27
10.1 Conventions	27
10.2 Key Endpoints	27
10.3 WebSocket Channels	28
10.4 Gateway Rules	28
11. Notifications	29
11.1 Event → Notification Mapping	29
11.2 Notification Queue	29
12. Infrastructure & Deployment	31
12.1 Initial Deployment Topology	31
12.2 Docker Compose for Local and Prod	31
12.3 Backups: The "Untested Backup Is A Hope" Rule	31
12.4 CI/CD Pipeline	32
12.5 Monitoring	32
12.6 When to Split Services	32
13. Architecture Decision Records (ADRs)	33
ADR-001: PostgreSQL + PostGIS as the Source of Truth	33
ADR-002: React Native + Expo for Mobile	33
ADR-003: Modular Monolith over Microservices	34
ADR-004: Adaptive GPS over Continuous Streaming	35

Appendix A: Pick-up Stations Reference

A.1 South Coast Stations

A.2 North Coast Stations

A.3 Train Schedules

A.4 Pricing Rules

A.5 Fare Calculation Example

36

36

37

37

38

1. Executive Summary

This document defines the production architecture, domain model, and database schema for the **SGR Feeder Platform** — a multi-user transport coordination system that connects passengers arriving at or departing from the Mombasa SGR Terminus with shared or private cab services operating along the North Coast and South Coast corridors.

The platform exists to solve a specific operational pain point that is currently absorbed informally: cab drivers at the existing SGR waiting/collection points (stages) cannot leave for the terminus until their vehicle is full, and "full" depends on strangers physically showing up at the stage in real time. That unpredictability eats into the check-in, ticketing, and security buffer that passengers need in order to catch their train. The same problem repeats in reverse for arriving passengers, who must find a cab home from a terminus that may be unfamiliar to them.

The platform converts both of these physical waiting problems into digital coordination problems. Passengers reserve seats in advance. Drivers see seat counts fill virtually, hours before anyone is physically at the stage. Departure times are reverse-engineered from the train schedule so that the cab leaves the pickup point at the latest possible moment that still gets passengers to the train on time. The return leg pools arriving passengers by destination zone, so a cab fills up by destination instead of by waiting at a stage.

1.1 Platform Scope

- **Passenger App** (React Native + Expo) — book shared or charter rides, choose pickup stage or off-stage location, pay via M-Pesa, receive live trip updates.
- **Driver App** (React Native + Expo) — receive assigned trips, view passenger manifest, send adaptive GPS updates, mark trip status changes.
- **Admin Web** (Next.js) — monitor arrivals and departures, detect stage coverage gaps, manage drivers and pricing, refund payments, audit operations.
- **Backend API** (NestJS + Fastify) — modular monolith exposing REST + WebSocket endpoints, owns all business rules and transactional integrity.
- **Database** (PostgreSQL + PostGIS) — single source of truth for all transactional state, with spatial indexes for location queries.

1.2 Architecture Principles

- **PostgreSQL is the source of truth. Redis is not.** Redis may hold ephemeral state (driver presence, recent GPS, rate limits) but never owns data the business cares about.
- **Mobile apps never talk directly to PostgreSQL.** All data flows through an authenticated API that enforces authorization, validation, and transactional integrity.
- **SQLite is an edge cache, not the centre.** Mobile devices use SQLite for offline profile/booking cache that syncs through the API. SQLite is not the authoritative database for this platform.
- **Modular monolith, not microservices.** One backend, one database, clean internal module boundaries. Service split happens only when the business outgrows the monolith — not on day one.

- **Bookings, rides, and trips are distinct entities.** A booking is a commercial reservation; a ride is a physical vehicle trip; multiple passengers can share a single ride. This separation is built in from day one.
- **State machines, not booleans.** Booking and payment status use explicit state machines with allowed transitions. Random boolean combinations like `isPaid && !isBooked` are impossible by construction.

1.3 Document Status

This is Version 1.0 of the engineering specification. It is intended as the authoritative reference for the founding engineering team. Subsequent revisions will be tracked via dated appendices; architectural changes that reverse decisions in this document must be recorded as new Architecture Decision Records (ADRs) in Chapter 13 rather than silently editing the existing text.

2. System Architecture Overview

The platform is structured as four independently deployable surfaces — two mobile apps, one web admin, and one backend — backed by a small set of infrastructure primitives. The diagram below shows the topology and the directional flow of trust. Every arrow represents an authenticated call; no client ever reads from or writes to the database directly.

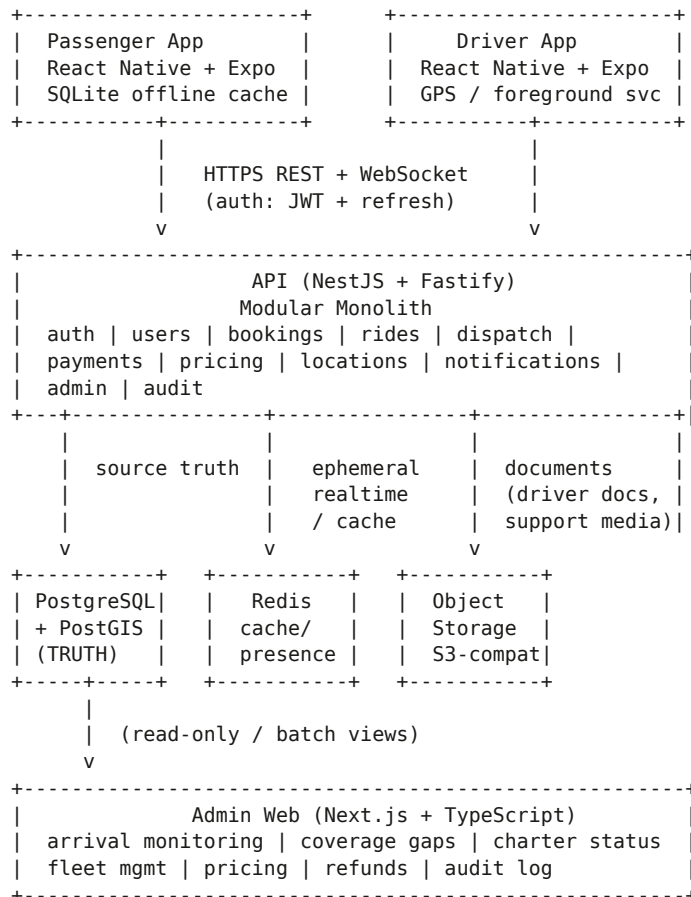


Figure 2.1 — Platform topology. PostgreSQL is the only source of truth.

2.1 The Trust Boundary

The API is the only component allowed to write to PostgreSQL. Mobile clients authenticate via short-lived JWT access tokens (15 minutes) backed by longer refresh tokens (rotated on each use). Admin operations require an additional RBAC role check at the service layer. No client receives database credentials, and no client is trusted to make authorization decisions — every isAdmin-style check happens inside the API, against the user's server-side role record.

The API also owns all rate limiting, audit logging, and idempotency enforcement. A duplicate M-Pesa callback, a double-tapped "book seat" button, or a flaky network retry must not produce duplicate bookings or duplicate charges. These guarantees are enforced at the database transaction boundary, not at the client.

2.2 Why Modular Monolith

A modular monolith runs as a single NestJS process with one PostgreSQL database, but internally it is organised into bounded modules: auth, bookings, rides, dispatch, payments, pricing, locations, notifications, admin, audit. Each module owns its own tables, service classes, and DTOs. Cross-module communication happens via explicit service calls, not via a shared database grab-bag.

This is the sweet spot for a startup that has not yet carried its first passenger. Microservices would saddle the team with twelve Docker containers, eight databases, five message brokers, and four deployment pipelines before the platform has any traffic to justify them. The modular monolith retains clean boundaries so that, when (and only when) a single module genuinely needs to scale independently — for example, the GPS ingestion pipeline — that module can be extracted into a separate service with minimal friction.

2.3 The Offline Edge

Mobile apps run on Kenyan mobile networks that are reliable in urban centres but uneven along the highway corridors that connect Mombasa to Malindi, Ukunda, and the Likoni South Coast. Each mobile app ships with a SQLite database that caches the user's profile, their most recent bookings, station lists, fare quotes, and any locally initiated actions that have not yet synced. The cache is authoritative for nothing — it is a read-through cache for offline display, and a write-ahead queue for actions that will be replayed when connectivity returns.

Critically, the SQLite cache never makes booking decisions locally. A passenger who taps "book seat" while offline sees a "queued for when you reconnect" state; the actual seat reservation happens server-side under a PostgreSQL transaction. This avoids the impossible problem of reconciling two passengers who both thought they had the last seat on a 4-seater while offline.

3. Technology Stack Decisions

Each layer of the stack is chosen for a concrete reason, not for fashion. The table below lists the chosen technology, the most credible alternative that was considered, and the reason for the choice. Where a decision is reversible later, that is noted; where it is effectively one-way (database, identity model), the rationale is correspondingly heavier.

Layer	Chosen	Alternative	Rationale
Passenger App	React Native + Expo + TypeScript	Flutter	Same TS ecosystem as the rest of the platform. Expo location tooling supports foreground services and Android 14 background-location permissions.
Driver App	React Native + Expo + TypeScript	Flutter	Same reasoning. Driver app shares ~70% of code with passenger app (auth, network layer, types).
Admin Web	Next.js + TypeScript	CRA / Vite SPA	SSR for fast first paint on admin dashboards. App Router suits the multi-section dashboard layout. Same TS ecosystem.
Backend	NestJS + Fastify + TypeScript	Express / Next.js API routes	Modular structure with DI, guards, interceptors. Fastify adapter is faster than Express. Far more maintainable than a giant Next.js API folder for a system this size.
Main DB	PostgreSQL + PostGIS	MongoDB / MySQL	ACID + row-level locking for booking concurrency. PostGIS for spatial queries (find drivers within X km of passenger). SQLite recommends client/server RDBMS for multi-writer systems.
Local Mobile DB	SQLite (via expo-sqlite)	Realm / WatermelonDB	Officially recommended by SQLite docs for local device storage. Simple, battle-tested, no sync engine lock-in.
Cache / Realtime	Redis	In-memory only	Driver presence, rate limiting, recent GPS, WebSocket scaling. NOT a source of truth. Can be skipped for initial low-traffic launch.
Payments	M-Pesa Daraja (with provider abstraction)	Direct M-Pesa hardcoding	Daraja 3.0 is the standard Kenyan payment API. Provider abstraction (payments/mpesa/ module) allows adding card / Airtel Money later.
Map UI	MapLibre	Google Maps SDK	Open-source, no per-load licensing cost. Map rendering separated from routing/geocoding.

Layer	Chosen	Alternative	Rationale
Routing	OSRM or Valhalla (self-hosted)	Google Directions API	Avoids per-request Google billing. Self-hosted on a single VPS handles MVP volume easily.
Object Storage	S3-compatible (e.g., MinIO self-hosted or Cloudflare R2)	Filesystem	Driver documents (ID, logbook, insurance) must survive server rebuilds. Filesystem ties data to one machine.
Deploy	Linux VPS + Docker Compose + Caddy	AWS ECS / Kubernetes	One VPS handles considerably more traffic than an MVP generates. Caddy gives automatic HTTPS. K8s is premature.
CI/CD	GitHub Actions	Jenkins / CircleCI	Native to GitHub repos, free for OSS, simple YAML.
Monitoring	Structured JSON logs + Sentry + UptimeRobot	ELK / Datadog	Structured logs are searchable via grep/jq. Sentry self-hosted for error tracking. UptimeRobot for external probes.

3.1 What This Stack Is Not

It is not a serverless architecture. Lambda-style functions introduce cold-start latency and obscure transaction boundaries — both poison for a booking system where the difference between "reserved" and "available" must be resolved in single-digit milliseconds. It is also not a NoSQL architecture. MongoDB would force us to rebuild transactional integrity in application code that PostgreSQL gives us for free, and we would lose PostGIS entirely — there is no equivalent document-store spatial index that approaches PostGIS for the queries this platform needs.

It is not a microservices architecture. The platform has not yet carried its first passenger. Adding a message broker, a service mesh, and inter-service network calls at this stage would burn engineering time on infrastructure instead of product. The modular monolith keeps the option open to extract services later — the module boundaries are the seams along which a future service split would happen.

4. Domain Model

The domain model is the most important decision in this document. Get it wrong and most of the application architecture becomes a fight against the data shape. The model below separates three concepts that are often incorrectly merged into a single "booking" table: the passenger's commercial reservation, the physical vehicle trip that fulfils it, and the operational route the vehicle takes with its stops.

4.1 Booking vs Ride vs Trip

A **booking** is what the passenger pays for. It is a commercial reservation: "I want a seat from Mombasa Terminus to Mtwapa on the 14:00 arriving train." The booking has a price, a payment state, and a passenger. It does not yet know which vehicle will carry it.

A **ride** is the physical vehicle trip that fulfils one or more bookings. A ride has a driver, a vehicle, a route, and a sequence of stops. When four passengers book seats on the same 4-seater cab going to Mtwapa, they all attach to the same ride. When one passenger books a private charter, they are the only booking on that ride, but the ride still exists as a distinct entity.

A **trip** in our domain model is the operational envelope around a ride: driver, vehicle, route, stops, and the dispatch state machine. We use "ride" and "trip" somewhat interchangeably in the codebase, but the database table is named rides to avoid collision with the transit-industry meaning of "trip" as a scheduled service.

Why the separation matters

If you collapse booking + ride into one table, you cannot represent shared rides without duplicating the driver, vehicle, and route across every passenger row. You also cannot represent a charter, where one booking consumes the whole vehicle. And you cannot reassign a passenger from one vehicle to another (e.g., when the original cab breaks down) without rewriting the booking. The separation lets all three cases work without schema changes.

4.2 Entity Inventory

Entity	Description
users	Identity root. Phone number is the natural key. One user can be passenger, driver, and admin simultaneously.
passenger_profiles	Passenger-specific data: saved pickup points, payment defaults, ratings received.
driver_profiles	Driver-specific data: license, NTSA logbook ref, verification status, vehicle assignment.
vehicles	Vehicle registry: plate, capacity, cab type (4/7/11/14-seater), insurance, owner.
driver_documents	Verification documents (ID, license, logbook, insurance). File in object storage; metadata in PostgreSQL.

Entity	Description
stations	Agreed SGR waiting/collection points used as pickup/drop-off stages (e.g., Kiembeni Mwisho, Likoni Ferry Container).
service_areas	Geographic polygons (South Coast, North Coast, Mombasa CBD) used for driver allocation and pricing.
pickup_points	Off-stage pickup locations (geocoded landmarks) with distance-from-nearest-stage for surcharge calculation.
journeys	A scheduled train departure or arrival. Ties to a specific train code and time.
bookings	Passenger's commercial reservation. Has price, payment state, lifecycle state machine.
rides	Physical vehicle trip. Has driver, vehicle, route, dispatch state. Multiple bookings attach to one ride.
ride_stops	Ordered stops on a ride (pickup and drop-off). Each stop has a location and a sequence number.
ride_passengers	Join table: which passengers are on which ride, which stop they board/alight at.
driver_assignments	Which driver is assigned to which ride, with assignment state (offered, accepted, rejected, revoked).
driver_locations	Live driver position (single row per driver, updated in place). For "driver is X km away" displays.
driver_location_events	Historical GPS events, partitioned by month. For trip replay, dispute resolution, analytics.
payments	Top-level payment record for a booking. Tracks total amount, provider, status.
payment_attempts	Individual attempts (STK push, callback). Idempotent on (provider, provider_transaction_id).
refunds	Refund records with reason, amount, approver, provider reference.
pricing_rules	Configurable fare rules: base fare, per-km surcharge, charter multiplier, peak-hour multiplier.
fare_quotes	Snapshot of a quoted fare (valid for X minutes) so the booking sees the same price the quote saw.
notifications	Outbound notification log: channel (push/SMS), payload, status, retry count.
ratings	Passenger-to-driver and driver-to-passenger ratings after a completed ride.
support_tickets	Customer support tickets linked to bookings, rides, or payments.
audit_logs	Append-only record of state-changing actions: actor, action, entity, metadata, IP, timestamp.

5. Database Schema (Full SQL DDL)

The schema below is presented as runnable PostgreSQL DDL with PostGIS extensions enabled. The DDL is organised by domain: identity, geography, operations, payments, location, pricing, and system. All foreign keys are explicit; all spatial columns use geography(Point,4326) so distance queries return metres without manual projection. Indexes are partial where appropriate (e.g., only active bookings are indexed by ride_id).

A few conventions: every table has created_at and updated_at timestamps managed by triggers (not shown for brevity). Every primary key is a UUID v7 (time-sortable, index-friendly). Enumerations are CHECK constraints rather than enum types so that schema migrations do not require enum type alterations. Soft-deleted rows have deleted_at set; queries filter WHERE deleted_at IS NULL via a partial index where it matters.

5.1 Extensions and Conventions

```
-- Required extensions
CREATE EXTENSION IF NOT EXISTS postgis;
CREATE EXTENSION IF NOT EXISTS "uuid-oss";
CREATE EXTENSION IF NOT EXISTS pg_trgm;      -- fuzzy text search for landmarks

-- UUID v7 generation function (time-sortable, index-friendly)
-- In production, use pg_uuidv7 extension or application-side generation.
CREATE OR REPLACE FUNCTION uuid_v7() RETURNS uuid AS $$
DECLARE
    unix_ts_ms bigint := extract(epoch from clock_timestamp()) * 1000;
    uuid_bytes bytea;
BEGIN
    uuid_bytes := decode(
        lpad(to_hex(unix_ts_ms), 12, '0') ||
        lpad(to_hex(floor(random() * 4294967296)::bigint), 8, '0') ||
        lpad(to_hex(floor(random() * 4294967296)::bigint), 8, '0') ||
        lpad(to_hex(floor(random() * 65536)::bigint), 4, '0'),
        'hex'
    );
    -- Set version 7 and variant bits per RFC 9562
    RETURN encode(uuid_bytes, 'hex')::uuid;
END;
$$ LANGUAGE plpgsql;

-- updated_at trigger (apply to every table)
CREATE OR REPLACE FUNCTION touch_updated_at()
RETURNS TRIGGER AS $$
BEGIN
    NEW.updated_at = clock_timestamp();
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

5.2 Identity & Profiles

```
CREATE TABLE users (
    id                uuid PRIMARY KEY DEFAULT uuid_v7(),
    phone             text NOT NULL UNIQUE,      -- E.164 format: +2547XXXXXXX
    phone_verified    boolean NOT NULL DEFAULT false,
    email             text UNIQUE,
    email_verified    boolean NOT NULL DEFAULT false,
    password_hash     text,                      -- admins only; passengers use OTP
    role_mask         integer NOT NULL DEFAULT 1, -- bitmask: 1=passenger 2=driver 4=admin
    status            text NOT NULL DEFAULT 'active'
    CHECK (status IN ('active','suspended','deactivated')),
```

```

    last_login_at    timestamptz,
    created_at       timestamptz NOT NULL DEFAULT clock_timestamp(),
    updated_at       timestamptz NOT NULL DEFAULT clock_timestamp(),
    deleted_at       timestamptz
);
CREATE INDEX users_role_mask_idx ON users (role_mask) WHERE deleted_at IS NULL;

CREATE TABLE passenger_profiles (
    id                uuid PRIMARY KEY DEFAULT uuid_v7(),
    user_id           uuid NOT NULL REFERENCES users(id),
    display_name      text NOT NULL,
    default_stage_id  uuid REFERENCES stations(id),
    rating_avg        numeric(2,1) NOT NULL DEFAULT 0.0,
    rating_count      integer NOT NULL DEFAULT 0,
    created_at        timestamptz NOT NULL DEFAULT clock_timestamp(),
    updated_at        timestamptz NOT NULL DEFAULT clock_timestamp(),
    UNIQUE (user_id)
);

CREATE TABLE driver_profiles (
    id                uuid PRIMARY KEY DEFAULT uuid_v7(),
    user_id           uuid NOT NULL REFERENCES users(id),
    display_name      text NOT NULL,
    license_number    text NOT NULL UNIQUE,
    ntsa_logbook_ref  text,                -- reference to vehicle logbook
    vehicle_id        uuid REFERENCES vehicles(id),
    verification_status text NOT NULL DEFAULT 'pending'
        CHECK (verification_status IN
            ('pending','submitted','verified','rejected','suspended')),
    approval_status   text NOT NULL DEFAULT 'pending'
        CHECK (approval_status IN
            ('pending','approved','rejected','revoked')),
    rating_avg        numeric(2,1) NOT NULL DEFAULT 0.0,
    rating_count      integer NOT NULL DEFAULT 0,
    created_at        timestamptz NOT NULL DEFAULT clock_timestamp(),
    updated_at        timestamptz NOT NULL DEFAULT clock_timestamp(),
    UNIQUE (user_id)
);
CREATE INDEX driver_profiles_verification_idx
    ON driver_profiles (verification_status, approval_status);

CREATE TABLE vehicles (
    id                uuid PRIMARY KEY DEFAULT uuid_v7(),
    plate_number      text NOT NULL UNIQUE,
    cab_type          text NOT NULL CHECK (cab_type IN
        ('4-seater','7-seater','11-seater','14-seater')),
    capacity          integer NOT NULL CHECK (capacity > 0 AND capacity <= 20),
    owner_user_id     uuid REFERENCES users(id),    -- may differ from driver
    insurance_expiry  date NOT NULL,
    inspection_expiry date NOT NULL,
    status            text NOT NULL DEFAULT 'active'
        CHECK (status IN ('active','maintenance','retired')),
    created_at        timestamptz NOT NULL DEFAULT clock_timestamp(),
    updated_at        timestamptz NOT NULL DEFAULT clock_timestamp()
);
CREATE INDEX vehicles_status_idx ON vehicles (status) WHERE status = 'active';

CREATE TABLE driver_documents (
    id                uuid PRIMARY KEY DEFAULT uuid_v7(),
    driver_profile_id uuid NOT NULL REFERENCES driver_profiles(id),
    doc_type          text NOT NULL CHECK (doc_type IN
        ('national_id','driving_license','logbook','insurance',
        'ntsa_inspection','passport_photo')),
    storage_key       text NOT NULL,                -- S3-compatible object key
    status            text NOT NULL DEFAULT 'pending'
        CHECK (status IN ('pending','verified','rejected','expired')),
    uploaded_at       timestamptz NOT NULL DEFAULT clock_timestamp(),
    verified_at        timestamptz,
    verified_by        uuid REFERENCES users(id),
    expires_at        date,

```

```

    metadata          jsonb NOT NULL DEFAULT '{}'::jsonb
);
CREATE INDEX driver_documents_driver_idx ON driver_documents (driver_profile_id, doc_type);

```

5.3 Geography (Stations, Service Areas, Pickup Points)

-- Note: stations table is referenced by passenger_profiles and others above.
 -- Forward declaration pattern: create stations before profiles in real migrations.

```

CREATE TABLE service_areas (
  id          uuid PRIMARY KEY DEFAULT uuid_v7(),
  name        text NOT NULL UNIQUE,          -- 'South Coast','North Coast','Mombasa CBD'
  coast       text NOT NULL CHECK (coast IN ('south','north','cbd')),
  boundary    geography(Polygon, 4326) NOT NULL,
  is_active   boolean NOT NULL DEFAULT true,
  created_at  timestampz NOT NULL DEFAULT clock_timestamp(),
  updated_at  timestampz NOT NULL DEFAULT clock_timestamp()
);
CREATE INDEX service_areas_boundary_gist_idx ON service_areas USING GIST (boundary);

CREATE TABLE stations (
  id          uuid PRIMARY KEY DEFAULT uuid_v7(),
  name        text NOT NULL,                  -- 'Kiambeni Mwisho','Likoni Ferry Container'
  area        text NOT NULL,                  -- 'Likoni','Bamburi','Mtwapa','Malindi','Kombani','Ukunda'
  coast       text NOT NULL CHECK (coast IN ('south','north','cbd')),
  location    geography(Point, 4326) NOT NULL,
  landmark    text,                          -- human-readable description
  travel_min_to_terminus integer NOT NULL,    -- typical travel time in minutes
  peak_adjust_min integer NOT NULL DEFAULT 0, -- extra minutes during peak hours
  is_active   boolean NOT NULL DEFAULT true,
  created_at  timestampz NOT NULL DEFAULT clock_timestamp(),
  updated_at  timestampz NOT NULL DEFAULT clock_timestamp(),
  UNIQUE (name, area)
);
CREATE INDEX stations_location_gist_idx ON stations USING GIST (location);
CREATE INDEX stations_active_idx ON stations (coast, area) WHERE is_active = true;

CREATE TABLE pickup_points (
  id          uuid PRIMARY KEY DEFAULT uuid_v7(),
  nearest_station_id uuid NOT NULL REFERENCES stations(id),
  label       text NOT NULL,                  -- 'Near Tuskys Bamburi'
  location    geography(Point, 4326) NOT NULL,
  distance_km numeric(4,2) NOT NULL,          -- precomputed distance to nearest station
  created_by  uuid REFERENCES users(id),      -- the passenger who first used it
  created_at  timestampz NOT NULL DEFAULT clock_timestamp()
);
CREATE INDEX pickup_points_location_gist_idx ON pickup_points USING GIST (location);
CREATE INDEX pickup_points_station_idx ON pickup_points (nearest_station_id);

```

5.4 Operations (Journeys, Bookings, Rides, Stops)

```

CREATE TABLE journeys (
  id          uuid PRIMARY KEY DEFAULT uuid_v7(),
  train_code  text NOT NULL,          -- 'Madaraka Express'
  train_time  time NOT NULL,          -- 08:00, 15:00, 22:00, 04:00, 14:00, 20:30
  direction   text NOT NULL CHECK (direction IN ('inbound','outbound')),
  origin      text NOT NULL,
  destination text NOT NULL,
  journey_date date NOT NULL,
  is_active   boolean NOT NULL DEFAULT true,
  created_at  timestampz NOT NULL DEFAULT clock_timestamp(),
  UNIQUE (train_code, train_time, journey_date)
);

CREATE TABLE bookings (
  id          uuid PRIMARY KEY DEFAULT uuid_v7(),
  booking_ref text NOT NULL UNIQUE,    -- human-readable: SGRF-AB12CD
  passenger_id uuid NOT NULL REFERENCES users(id),
  journey_id  uuid NOT NULL REFERENCES journeys(id),
  pickup_kind text NOT NULL CHECK (pickup_kind IN ('stage','off-stage')),
  pickup_station_id uuid REFERENCES stations(id),
  pickup_point_id uuid REFERENCES pickup_points(id),
  dropoff_station_id uuid REFERENCES stations(id),
  seats_reserved integer NOT NULL CHECK (seats_reserved >= 1 AND seats_reserved <= 20),
  booking_kind  text NOT NULL CHECK (booking_kind IN ('pooled','charter')),
  state         text NOT NULL DEFAULT 'quote_created'
              CHECK (state IN
                ('quote_created','booking_pending','awaiting_payment',
                 'confirmed','assignment_pending','driver_assigned',
                 'driver_en_route','passenger_picked_up','in_progress',
                 'completed','expired','cancelled','no_show')),
  fare_quoted_ksh integer NOT NULL,    -- snapshot of fare at quote time
  fare_final_ksh  integer,             -- settled on completion
  -- CHECK constraint: charter must reserve all vehicle seats; pooled 1..N
  -- (enforced at application layer once ride is assigned)
  has_e_ticket    boolean NOT NULL DEFAULT false,
  confirmed_pickup_time timestampz,    -- reverse-engineered latest-leave
  ride_id         uuid REFERENCES rides(id), -- null until dispatch assigns
  created_at      timestampz NOT NULL DEFAULT clock_timestamp(),
  updated_at      timestampz NOT NULL DEFAULT clock_timestamp(),
  expires_at      timestampz NOT NULL,   -- quote expires in 15 min
  cancelled_at     timestampz,
  cancellation_reason text
);

CREATE INDEX bookings_passenger_idx ON bookings (passenger_id) WHERE cancelled_at IS NULL;
CREATE INDEX bookings_ride_idx ON bookings (ride_id) WHERE ride_id IS NOT NULL;
CREATE INDEX bookings_journey_state_idx ON bookings (journey_id, state)
  WHERE cancelled_at IS NULL;
CREATE INDEX bookings_state_idx ON bookings (state) WHERE cancelled_at IS NULL;

CREATE TABLE rides (
  id          uuid PRIMARY KEY DEFAULT uuid_v7(),
  journey_id  uuid NOT NULL REFERENCES journeys(id),
  driver_id   uuid REFERENCES driver_profiles(id),
  vehicle_id  uuid REFERENCES vehicles(id),
  stage_id    uuid NOT NULL REFERENCES stations(id), -- primary stage
  direction   text NOT NULL CHECK (direction IN ('inbound','outbound')),
  capacity_total integer NOT NULL,
  capacity_booked integer NOT NULL DEFAULT 0,
  is_charter   boolean NOT NULL DEFAULT false,
  state       text NOT NULL DEFAULT 'open'
              CHECK (state IN
                ('open','filling','locked','departed','completed','cancelled')),
  base_fare_ksh integer NOT NULL DEFAULT 450,
  current_fare_ksh integer NOT NULL DEFAULT 450,
  departed_at  timestampz,
  completed_at timestampz,
  created_at   timestampz NOT NULL DEFAULT clock_timestamp(),

```



```

    updated_at      timestampz NOT NULL DEFAULT clock_timestamp()
);
CREATE INDEX rides_journey_idx ON rides (journey_id, direction, state);
CREATE INDEX rides_driver_idx ON rides (driver_id) WHERE state IN ('open','filling','locked','departed');
CREATE INDEX rides_stage_idx ON rides (stage_id, direction, state);

CREATE TABLE ride_stops (
  id                uuid PRIMARY KEY DEFAULT uuid_v7(),
  ride_id           uuid NOT NULL REFERENCES rides(id),
  sequence          integer NOT NULL,           -- 1, 2, 3...
  stop_type         text NOT NULL CHECK (stop_type IN ('pickup','dropoff')),
  station_id        uuid REFERENCES stations(id),
  pickup_point_id   uuid REFERENCES pickup_points(id),
  location          geography(Point, 4326) NOT NULL,
  planned_time      timestampz,                -- reverse-engineered
  actual_time       timestampz,
  created_at        timestampz NOT NULL DEFAULT clock_timestamp(),
  UNIQUE (ride_id, sequence)
);
CREATE INDEX ride_stops_ride_idx ON ride_stops (ride_id, sequence);

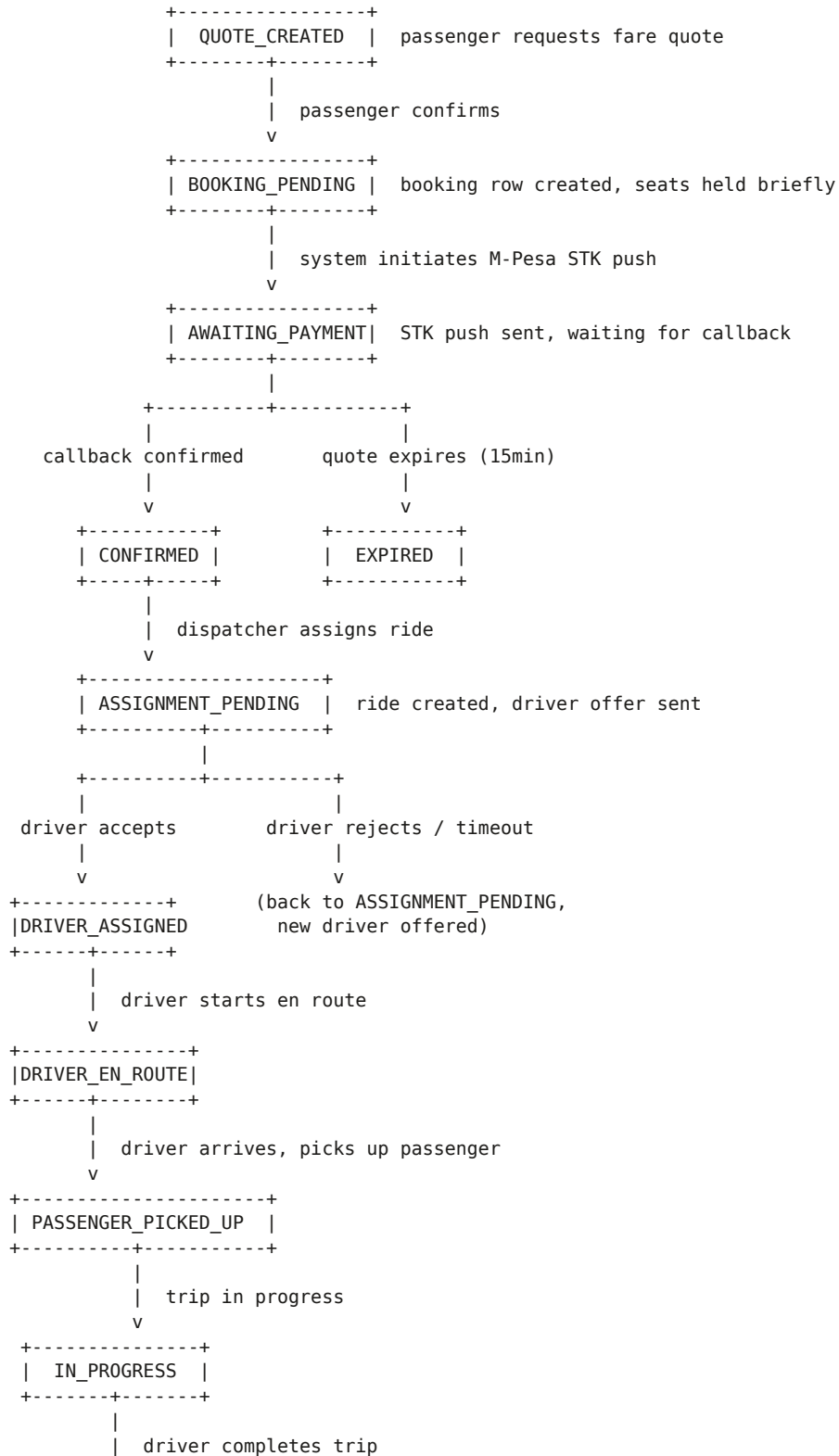
CREATE TABLE ride_passengers (
  id                uuid PRIMARY KEY DEFAULT uuid_v7(),
  ride_id           uuid NOT NULL REFERENCES rides(id),
  booking_id        uuid NOT NULL REFERENCES bookings(id),
  board_stop_id     uuid NOT NULL REFERENCES ride_stops(id),
  alight_stop_id    uuid NOT NULL REFERENCES ride_stops(id),
  seats             integer NOT NULL CHECK (seats >= 1),
  created_at        timestampz NOT NULL DEFAULT clock_timestamp(),
  UNIQUE (booking_id),                        -- one booking = one ride row
  UNIQUE (ride_id, booking_id)
);
CREATE INDEX ride_passengers_ride_idx ON ride_passengers (ride_id);

CREATE TABLE driver_assignments (
  id                uuid PRIMARY KEY DEFAULT uuid_v7(),
  ride_id           uuid NOT NULL REFERENCES rides(id),
  driver_id         uuid NOT NULL REFERENCES driver_profiles(id),
  state             text NOT NULL DEFAULT 'offered'
                    CHECK (state IN ('offered','accepted','rejected','revoked','completed')),
  offered_at        timestampz NOT NULL DEFAULT clock_timestamp(),
  responded_at      timestampz,
  revoked_at        timestampz,
  revoked_reason    text,
  UNIQUE (ride_id)                                -- one driver per ride at a time
);
CREATE INDEX driver_assignments_driver_state_idx
  ON driver_assignments (driver_id, state) WHERE state IN ('offered','accepted');

```

6. Booking State Machine

Booking state is modelled as an explicit finite state machine, not as a collection of booleans. This eliminates impossible combinations (e.g., `isPaid && isCancelled`) by construction and makes the lifecycle visible to engineers, ops, and support. Each transition is guarded by an allowed-actor check and produces audit-log entries plus side effects.



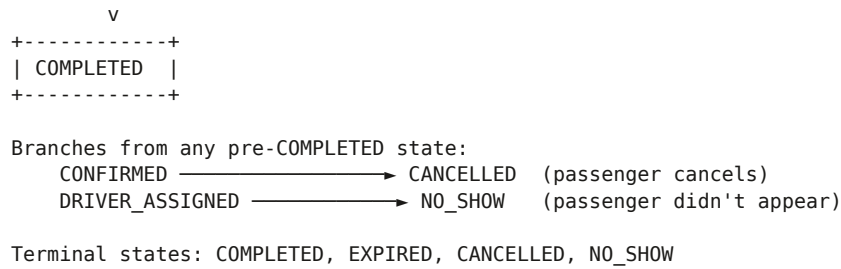


Figure 6.1 — Booking lifecycle state machine.

6.1 Transition Rules

From State	To State	Actor	Trigger & Side Effects
QUOTE_CREATED	BOOKING_PENDING	Passenger	Quote created, passenger confirms seat selection.
BOOKING_PENDING	AWAITING_PAYMENT	System	M-Pesa STK push initiated, payment_attempt row created.
BOOKING_PENDING	EXPIRED	System (timer)	15-minute quote TTL elapsed without confirmation.
AWAITING_PAYMENT	CONFIRMED	System (callback)	M-Pesa callback verified, payment recorded.
AWAITING_PAYMENT	EXPIRED	System (timer)	Payment callback not received within 5 minutes.
CONFIRMED	ASSIGNMENT_PENDING	System (dispatcher)	Ride created, driver assignment offered.
CONFIRMED	CANCELLED	Passenger / Admin	Pre-departure cancellation, triggers refund flow.
ASSIGNMENT_PENDING	DRIVER_ASSIGNED	Driver (accept)	Driver accepts assignment via app.
ASSIGNMENT_PENDING	ASSIGNMENT_PENDING	System (timeout)	Driver rejected or timed out; new driver offered.
DRIVER_ASSIGNED	DRIVER_ENROUTE	Driver	Driver taps "Start trip" in app.
DRIVER_ASSIGNED	NO_SHOW	Driver / System	Passenger not at pickup after 15min grace.
DRIVER_ENROUTE	PASSENGER_PICKED_UP	Driver	Driver confirms pickup at stop.
PASSENGER_PICKED_UP	IN_PROGRESS	Driver	Vehicle departs pickup location.
IN_PROGRESS	COMPLETED	Driver	Vehicle arrives at drop-off, all passengers alight.

Table 6.1 — Allowed state transitions. Any transition not listed is rejected by the API.

6.2 Concurrency: Holding Seats Without Selling Them Twice

When two passengers tap "book seat" on the same 4-seater cab at nearly the same instant, only one can get the last seat. PostgreSQL's row-level locking resolves this with `SELECT ... FOR UPDATE` on the rides row inside a transaction. The full booking transaction looks like:

```
BEGIN;

-- Lock the ride row; other concurrent bookings block here
SELECT capacity_total, capacity_booked
  FROM rides
 WHERE id = :ride_id
  FOR UPDATE;

-- Verify availability (re-check after lock acquired)
IF capacity_booked + :seats_requested > capacity_total THEN
  ROLLBACK;
  RETURN insufficient_capacity;
END IF;

-- Create the booking row
INSERT INTO bookings (...) VALUES (...);

-- Atomically reserve capacity
UPDATE rides
  SET capacity_booked = capacity_booked + :seats_requested
 WHERE id = :ride_id;

-- Create the payment attempt (idempotency key prevents dup STK push)
INSERT INTO payment_attempts (...) VALUES (...);

COMMIT;
```

The `FOR UPDATE` lock is held for milliseconds. If Passenger B's transaction begins while Passenger A's is in flight, B waits until A commits, then re-reads the capacity and either succeeds (if A took fewer seats than the remaining capacity) or fails cleanly. There is no window in which both passengers can be sold the same seat.

For high-contention scenarios (e.g., a popular train time with multiple cabs filling at once), we can use `SKIP LOCKED` to skip over locked rides and try the next available cab in the queue, rather than blocking. This pattern is documented in PostgreSQL's `SELECT` reference and is the standard way to build work-queue-style dispatch without Redis.

7. Payment Flows

Payments are integrated with M-Pesa Daraja 3.0 (Safaricom's developer platform) but the payment module is structured as a provider abstraction so we can add Airtel Money, card processing, or corporate billing later without leaking provider concepts into the booking module.

7.1 Payment Lifecycle

A payment is never considered successful just because M-Pesa sent a callback. The flow verifies every callback against the provider's transaction query API before marking the booking as paid. This protects against spoofed callbacks and against Daraja's occasional duplicate delivery of the same callback.

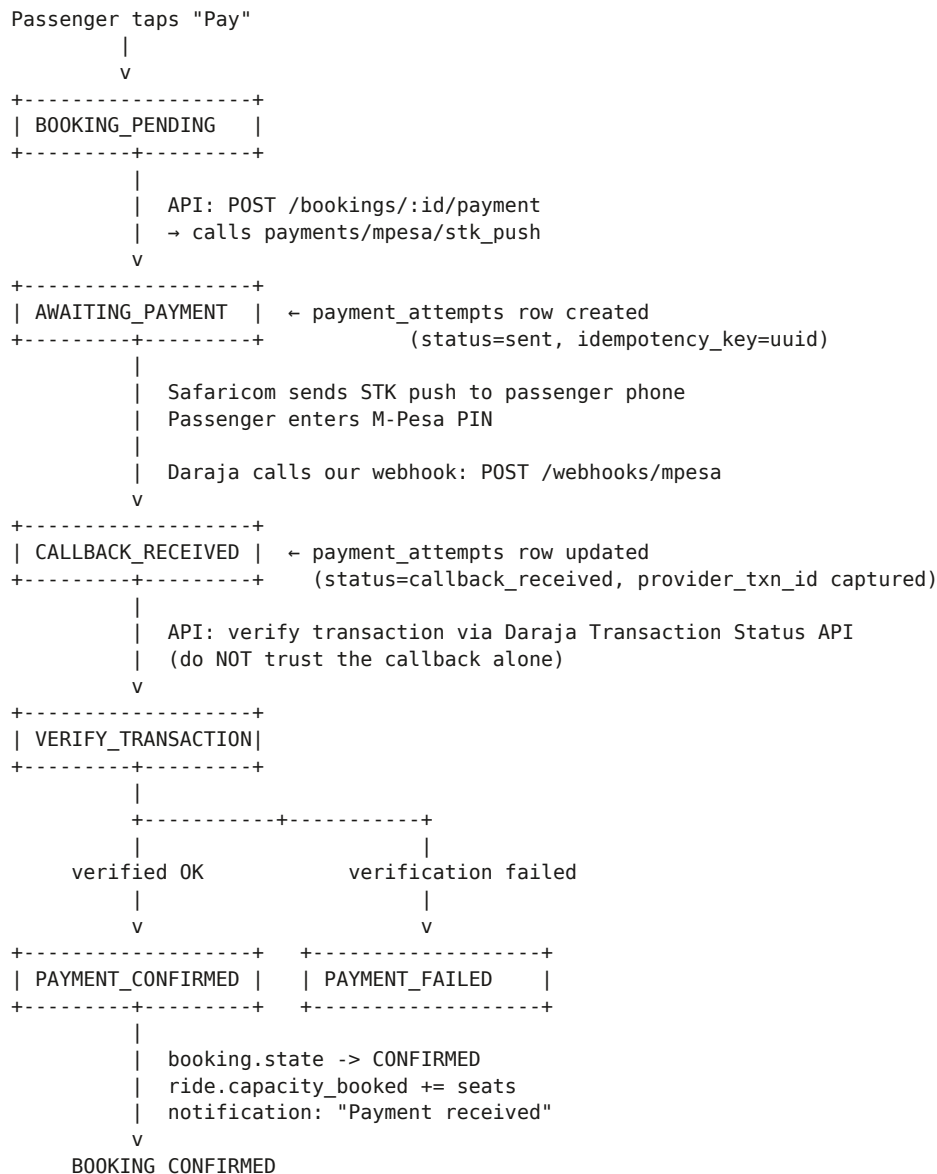


Figure 7.1 — Payment verification flow. Callbacks are never trusted alone.

7.2 Idempotency

Safaricom occasionally delivers the same callback twice. Without idempotency, the second callback would confirm a second payment on a booking that was already paid. The schema enforces idempotency at two levels: (1) the `payment_attempts` table has a unique constraint on `(provider, provider_transaction_id)`, so the second insert fails; and (2) the booking state machine refuses the `AWAITING_PAYMENT -> CONFIRMED` transition once the booking is already confirmed.

7.3 Payment Schema (DDL)

```
CREATE TABLE payments (
  id          uuid PRIMARY KEY DEFAULT uuid_v7(),
  booking_id  uuid NOT NULL REFERENCES bookings(id),
  amount_ksh  integer NOT NULL CHECK (amount_ksh > 0),
  currency    text NOT NULL DEFAULT 'KES',
  provider    text NOT NULL DEFAULT 'mpesa'
              CHECK (provider IN ('mpesa', 'airtel', 'card', 'cash', 'corporate')),
  provider_payment_ref text,          -- M-Pesa transaction ID
  status      text NOT NULL DEFAULT 'pending'
              CHECK (status IN ('pending', 'confirmed', 'failed', 'refunded', 'partially_refunded')),
  created_at  timestampz NOT NULL DEFAULT clock_timestamp(),
  updated_at  timestampz NOT NULL DEFAULT clock_timestamp(),
  UNIQUE (booking_id)                -- one payment per booking
);

CREATE TABLE payment_attempts (
  id          uuid PRIMARY KEY DEFAULT uuid_v7(),
  payment_id  uuid NOT NULL REFERENCES payments(id),
  provider    text NOT NULL,
  idempotency_key text NOT NULL,      -- client-supplied UUID
  provider_request_ref text,          -- M-Pesa CheckoutRequestID
  provider_transaction_id text,       -- M-Pesa MpesaCode (from callback)
  status      text NOT NULL DEFAULT 'sent'
              CHECK (status IN ('sent', 'callback_received', 'verified', 'failed', 'timeout')),
  request_payload jsonb NOT NULL,
  response_payload jsonb,
  callback_payload jsonb,
  sent_at     timestampz NOT NULL DEFAULT clock_timestamp(),
  callback_at timestampz,
  verified_at timestampz,
  UNIQUE (provider, provider_transaction_id) -- ← idempotency on callback
);
CREATE INDEX payment_attempts_payment_idx ON payment_attempts (payment_id);
CREATE INDEX payment_attempts_idem_idx ON payment_attempts (idempotency_key);

CREATE TABLE refunds (
  id          uuid PRIMARY KEY DEFAULT uuid_v7(),
  payment_id  uuid NOT NULL REFERENCES payments(id),
  amount_ksh  integer NOT NULL CHECK (amount_ksh > 0),
  reason      text NOT NULL,
  provider_ref text,
  status      text NOT NULL DEFAULT 'pending'
              CHECK (status IN ('pending', 'completed', 'failed')),
  approved_by uuid NOT NULL REFERENCES users(id), -- admin
  approved_at timestampz NOT NULL DEFAULT clock_timestamp(),
  completed_at timestampz,
  UNIQUE (payment_id)                -- at most one refund per payment
);
```

7.4 Provider Abstraction

The payment module exposes a `PaymentProvider` interface with methods like `initiateCharge(booking, payment_attempt)` and `verifyCallback(payload) -> { verified, transactionId, amount }`. The M-Pesa

implementation lives in `payments/mpesa/`. When we add Airtel Money or card processing, each gets its own subdirectory implementing the same interface. The booking module never sees provider-specific concepts like "STK push" or "CheckoutRequestID" — it only sees the abstract payment state transitions.

8. Location Tracking Strategy

Driver location tracking is the most operationally complex piece of the platform. Naively streaming one GPS update per second per active driver would saturate the API, bloat the database, drain driver phone batteries, and run into Android's background-location restrictions on Android 14 and later. The strategy below treats GPS as a state-dependent signal whose frequency adapts to what the platform actually needs to know about each driver at any given moment.

8.1 Two Concepts: Current Position vs Historical Events

A driver has two distinct location records. The **current position** is a single row per driver, updated in place, used to answer "where is the driver right now?" for passenger-facing ETAs and admin dispatch dashboards. The **historical events** are an append-only log, partitioned by month, used for trip replay, dispute resolution, route analysis, and operational analytics. Conflating these into one table creates either a write-amplification problem (every GPS ping updates the "current" row AND inserts a history row) or a query problem (history queries scan the live table).

```
-- Live position (one row per driver, updated in place)
CREATE TABLE driver_locations (
  driver_profile_id uuid PRIMARY KEY REFERENCES driver_profiles(id),
  location          geography(Point, 4326) NOT NULL,
  heading_deg       numeric(4,1),           -- 0..359.9
  speed_kmh         numeric(4,1),
  accuracy_m        integer,                -- GPS reported accuracy
  recorded_at       timestamptz NOT NULL,
  updated_at        timestamptz NOT NULL DEFAULT clock_timestamp()
);
CREATE INDEX driver_locations_gist_idx ON driver_locations USING GIST (location);

-- Historical events (append-only, partitioned by month)
CREATE TABLE driver_location_events (
  id                uuid NOT NULL DEFAULT uuid_v7(),
  driver_profile_id uuid NOT NULL REFERENCES driver_profiles(id),
  ride_id           uuid REFERENCES rides(id),    -- null when not on a ride
  location          geography(Point, 4326) NOT NULL,
  heading_deg       numeric(4,1),
  speed_kmh         numeric(4,1),
  accuracy_m        integer,
  battery_pct       integer,                -- for adaptive frequency
  recorded_at       timestamptz NOT NULL,
  PRIMARY KEY (id, recorded_at)
) PARTITION BY RANGE (recorded_at);

-- Create monthly partitions (example for 2026-08)
CREATE TABLE driver_location_events_2026_08
  PARTITION OF driver_location_events
  FOR VALUES FROM ('2026-08-01') TO ('2026-09-01');
CREATE INDEX dle_2026_08_driver_time_idx
  ON driver_location_events_2026_08 (driver_profile_id, recorded_at DESC);
CREATE INDEX dle_2026_08_ride_idx
  ON driver_location_events_2026_08 (ride_id) WHERE ride_id IS NOT NULL;
```

8.2 Adaptive GPS Frequency

The driver app does not send GPS at a fixed interval. Instead, it adapts based on the driver's current operational state, the device's battery level, and a distance threshold (only send if moved >50m from last reported position). This dramatically reduces bandwidth and database writes while still giving the platform the

resolution it needs for each operational context.

Driver State	GPS Frequency	Rationale
Offline / off-duty	0 (no tracking)	Driver is not working; no location collected.
Online, idle (no assignment)	Every 5 minutes	Lets admin see roughly where idle drivers are for dispatch.
Assignment offered, not accepted	Every 5 minutes	Driver considering; no urgency.
Assigned to pickup (en route to passenger)	Every 15 seconds	Passenger needs to see driver approaching; live ETA matters.
Passenger picked up (trip in progress)	Every 10 seconds	Trip replay resolution; safety monitoring.
Trip completed, awaiting next assignment	Every 5 minutes	Back to idle state.
Battery < 15%	Half the normal frequency	Preserve battery; degrade gracefully.

Table 8.1 — Adaptive GPS frequency. Combine with 50m minimum-distance threshold.

8.3 Android Background Location Reality

Expo's location tooling supports foreground and background location, Android foreground services, and geofencing — but background location has OS-level limitations that the driver app must respect. Android 14 (API 34) requires the `FOREGROUND_SERVICE_LOCATION` permission for any foreground service that accesses location, and the driver app must display a persistent notification while such a service is running. iOS imposes its own 30-second background task windows that are unsuitable for continuous tracking.

Practical implication: while the driver app is in the foreground (driver is actively looking at their next pickup), we get high-frequency updates cheaply. When the app goes to background, we rely on a foreground service (Android) to keep location flowing — but we still degrade the frequency aggressively because the OS will kill services that drain battery. We do NOT design the system around assuming the driver's phone will continuously stream GPS every second for their entire working day.

9. Security & Authentication

Security is layered: HTTPS at the transport, JWT-based authentication at the application, role-based access control (RBAC) at the service, and row-level policies inside PostgreSQL for defence in depth. Authorization is enforced on the server, never on the client — the client can lie about who it is, but the API always re-checks against the canonical user record.

9.1 Authentication

Passengers authenticate with phone number + OTP. This is the expected pattern for a Kenyan consumer app and avoids password fatigue. Drivers authenticate with phone + OTP plus an additional device/session binding step (the driver app is bound to one device at a time; a new device login forces a re-verification of the driver's identity). Admins authenticate with phone + OTP plus a password, and admin sessions have shorter refresh-token lifetimes.

All sessions issue a short-lived JWT access token (15 minutes) plus a longer refresh token (7 days, rotated on each use). Refresh tokens are stored hashed in a `refresh_tokens` table so they can be revoked. On logout, on password change, or on suspicious activity, all refresh tokens for that user are invalidated.

9.2 Role Hierarchy

A user's row has a `role_mask` bitmask: 1=passenger, 2=driver, 4=admin. A single user can hold multiple roles (e.g., a driver who also books personal trips as a passenger). The role profiles live in separate tables (`passenger_profiles`, `driver_profiles`) that reference back to the user. There is one authentication system; there are three authorization profiles.

9.3 Authorization Matrix

Action	Passenger	Driver	Admin	Endpoint
Read own bookings	Yes	No	Yes (own + filtered)	GET /bookings?me=true
Create booking	Yes	No	No	POST /bookings
Cancel own booking	Yes	No	Yes (any booking)	POST /bookings/:id/cancel
Read assigned trips	No	Yes	Yes (any trip)	GET /trips?driver=:me
Update trip status	No	Yes (own trips only)	Yes	PATCH /trips/:id/status
Send driver location	No	Yes (own)	No	POST /drivers/:me/location
Manage drivers	No	No	Yes	POST /admin/drivers

Action	Passenger	Driver	Admin	Endpoint
Manage pricing rules	No	No	Yes	PUT /admin/pricing
Refund payments	No	No	Yes	POST /admin/refunds
View audit logs	No	No	Yes	GET /admin/audit-logs

Table 9.1 — Authorization matrix. Every cell is enforced server-side.

9.4 Driver Verification

Drivers carry passengers. The platform cannot allow an unverified driver to accept bookings. The `driver_profiles` table tracks `verification_status` (documents submitted, verified, rejected) separately from `approval_status` (the admin's operational approval to start driving). A driver must be in verified + approved state before any ride can be assigned to them.

Verification documents (national ID, driving license, vehicle logbook, insurance, NTSA inspection) are uploaded to S3-compatible object storage. PostgreSQL stores only the `storage_key`, document type, status, and metadata. The actual file never lives in the database. This keeps the database small, makes document rotation trivial (replace the S3 object, update the metadata), and allows documents to be served via signed URLs with short expiry for admin review.

9.5 Audit Logging

Every state-changing action — admin changes driver status, driver accepts trip, payment callback confirms, fare manually overridden — writes a row to `audit_logs`. This is operational gold the moment a dispute arises ("the driver said the passenger never showed, the passenger said they waited 30 minutes") and is mandatory for any platform handling money.

```
CREATE TABLE audit_logs (
  id          uuid PRIMARY KEY DEFAULT uuid_v7(),
  actor_id    uuid REFERENCES users(id),      -- null for system actions
  actor_role  text NOT NULL,                  -- 'passenger', 'driver', 'admin', 'system'
  action      text NOT NULL,                  -- 'booking.cancel', 'payment.refund', 'driver.suspend'
  entity_type text NOT NULL,                  -- 'booking', 'payment', 'driver_profile'
  entity_id   uuid,
  metadata    jsonb NOT NULL DEFAULT '{}'::jsonb,
  ip_address  inet,
  user_agent  text,
  created_at  timestampz NOT NULL DEFAULT clock_timestamp()
);
CREATE INDEX audit_logs_actor_idx ON audit_logs (actor_id, created_at DESC);
CREATE INDEX audit_logs_entity_idx ON audit_logs (entity_type, entity_id, created_at DESC);
CREATE INDEX audit_logs_action_idx ON audit_logs (action, created_at DESC);
```

Audit logs are append-only. No row is ever updated or deleted; corrections are made by writing a new audit entry that supersedes the prior one (chained via `metadata.supersedes_id`). Logs are retained for at least 7 years to satisfy Kenyan financial-services record-keeping norms.

10. API Contract

The API is REST with versioning in the URL path (/api/v1/...). Where real-time updates are required (driver location during a trip, dispatch offers to drivers, trip status updates to passengers), WebSocket connections are used as a complement — not a replacement — for the REST surface.

10.1 Conventions

- **Versioning:** /api/v1/... in URL path. Breaking changes bump version; non-breaking changes are additive.
- **Auth:** Authorization: Bearer <jwt> header on every request except /auth/* and webhook endpoints.
- **Idempotency:** All POST endpoints accept Idempotency-Key header. Same key + same payload returns the original response instead of duplicating the side effect.
- **Pagination:** ?cursor=...&limit=20 cursor-based pagination for list endpoints.
- **Errors:** RFC 7807 problem+json format. { "type": "...", "title": "...", "status": 409, "detail": "...", "instance": "/api/v1/bookings/123" }
- **Time:** All timestamps are ISO 8601 with timezone offset (2026-08-19T15:00:00+03:00). EAT throughout.
- **Money:** All amounts are integer KSh (no decimals). KSh 450.50 does not exist in this system.

10.2 Key Endpoints

Meth od	Path	Role	Description
POS T	/quotes	Passenger	Compute fare quote (pickup, dropoff, kind, seats). Returns quote_id valid 15min.
POS T	/bookings	Passenger	Create booking from quote. Initiates payment flow.
GET	/bookings/:id	Passenger/Dri ver/Admin	Fetch booking detail. Authorization scoped by role.
POS T	/bookings/:id/cancel	Passenger/Ad min	Cancel booking. Triggers refund if applicable.
POS T	/bookings/:id/payment	Passenger	Initiate M-Pesa STK push. Returns payment_attempt_id.
POS T	/webhooks/mpesa	M-Pesa	Daraja callback. Verifies and confirms payment. NO AUTH — verified by signature.
GET	/trips/assigned	Driver	List trips assigned to current driver.
GET	/trips/:id	Driver/Admin	Trip detail with manifest, stops, vehicle, driver.

Meth od	Path	Role	Description
PAT CH	/trips/:id/status	Driver	Update trip state (en_route, picked_up, in_progress, completed).
POS T	/drivers/me/location	Driver	Submit GPS update. Adaptive frequency enforced client-side.
POS T	/drivers/me/assignments/:id/accept	Driver	Accept a dispatch offer.
POS T	/drivers/me/assignments/:id/reject	Driver	Reject a dispatch offer.
GET	/admin/arrivals/:train_id	Admin	Stage-by-stage breakdown for an arriving train.
GET	/admin/stage-coverage	Admin	Full stage coverage map with gap detection.
POS T	/admin/refunds	Admin	Initiate refund for a payment.
GET	/admin/audit-logs	Admin	Query audit logs with filters.

Table 10.1 — Key REST endpoints. Full OpenAPI spec lives in /docs/api alongside this document.

10.3 WebSocket Channels

WebSocket connections are authenticated with the JWT in the initial handshake (query string ?token=... for browser clients, header for native clients). Two channels are defined today; more will be added as the platform matures.

Channel	Client	Purpose
/ws/driver	Driver	Server pushes dispatch offers, ride state changes, passenger cancellations. Driver acks offers via REST, not WS.
/ws/passenger/:booking_id	Passenger	Server pushes trip state updates for a specific booking (driver assigned, en route, picked up, completed).

10.4 Gateway Rules

The platform runs behind a single Caddy reverse proxy. The Caddyfile maps paths to backend services. For mini-services (e.g., the location ingestion worker that may eventually split out of the main API), cross-origin requests carry the target port in a XTransformPort query parameter. API request URLs are always relative paths — never absolute URLs with embedded ports. The same applies to WebSocket URLs: io("/?XTransformPort=3030"), never io("http://localhost:3030").

11. Notifications

The notification strategy is push-first, SMS-fallback. Push notifications are essentially free at volume (Expo Notifications handles FCM/APNs delivery) and arrive within seconds. SMS costs real money per message and should be reserved for moments where push has failed or the message is too critical to risk a missed delivery.

11.1 Event → Notification Mapping

Event	Channel	Template
Booking confirmed	Push	"Your seat is confirmed. Driver will be assigned shortly."
Driver assigned	Push	"Driver [name] assigned to your trip. Plate [plate]. Rating [rating]."
Driver en route	Push + SMS	"Driver is en route. ETA [X] minutes." Critical, so SMS backs up push.
Pickup time changed	Push + SMS	"Your pickup time changed to [time]. Be at [stage] by [time]."
Payment received	Push	"Payment of KSh [amount] received for booking [ref]."
Trip completed	Push	"Your trip is complete. Rate your driver?"
Trip cancelled	Push + SMS	"Your trip was cancelled. Reason: [reason]. Refund: [status]."
Charter accepted	Push	"[Driver] accepted your private charter. Whole vehicle reserved."
Coverage gap alert	Push (admin)	"Stage [name] has [N] passengers waiting with no cab assigned."

11.2 Notification Queue

Notifications are not sent synchronously from the action that triggered them. The action writes a row to the notifications table with status queued; a background worker picks up queued rows, attempts delivery, and updates status to sent or failed. Failed push notifications trigger an SMS fallback after the configured retry window. This decouples user-facing latency from notification provider latency.

```
CREATE TABLE notifications (
  id                uuid PRIMARY KEY DEFAULT uuid_v7(),
  user_id           uuid NOT NULL REFERENCES users(id),
  channel           text NOT NULL CHECK (channel IN ('push', 'sms', 'in_app')),
  event_type        text NOT NULL,           -- 'driver_assigned', 'trip_cancelled'...
  title             text NOT NULL,
  body             text NOT NULL,
  payload           jsonb NOT NULL DEFAULT '{}':::jsonb,
  status            text NOT NULL DEFAULT 'queued'
                  CHECK (status IN ('queued', 'sent', 'failed', 'superseded')),
  attempts          integer NOT NULL DEFAULT 0,
  max_attempts      integer NOT NULL DEFAULT 3,
  next_attempt_at   timestampz NOT NULL DEFAULT clock_timestamp(),
```

```
    sent_at          timestampz,  
    failed_at        timestampz,  
    failure_reason    text,  
    provider_message_id text,          -- FCM/APNs/SMS gateway message ID  
    created_at        timestampz NOT NULL DEFAULT clock_timestamp(),  
    updated_at        timestampz NOT NULL DEFAULT clock_timestamp()  
);  
CREATE INDEX notifications_queued_idx  
    ON notifications (next_attempt_at) WHERE status = 'queued';  
CREATE INDEX notifications_user_idx ON notifications (user_id, created_at DESC);
```

Push delivery uses Expo Notifications (which handles FCM for Android and APNs for iOS). SMS delivery uses Africa's Talking or a similar Kenyan SMS gateway. The provider abstraction is similar to the payments module — each provider implements a NotificationChannel interface, and adding a new SMS gateway does not touch the notification domain logic.

12. Infrastructure & Deployment

The platform starts on a single Linux VPS. This is deliberate. A startup that has not yet carried its first passenger does not need Kubernetes, a service mesh, or a multi-region deployment. What it needs is a system that is easy to operate, easy to back up, and easy to debug — all of which favour a single well-configured machine over a distributed fleet.

12.1 Initial Deployment Topology

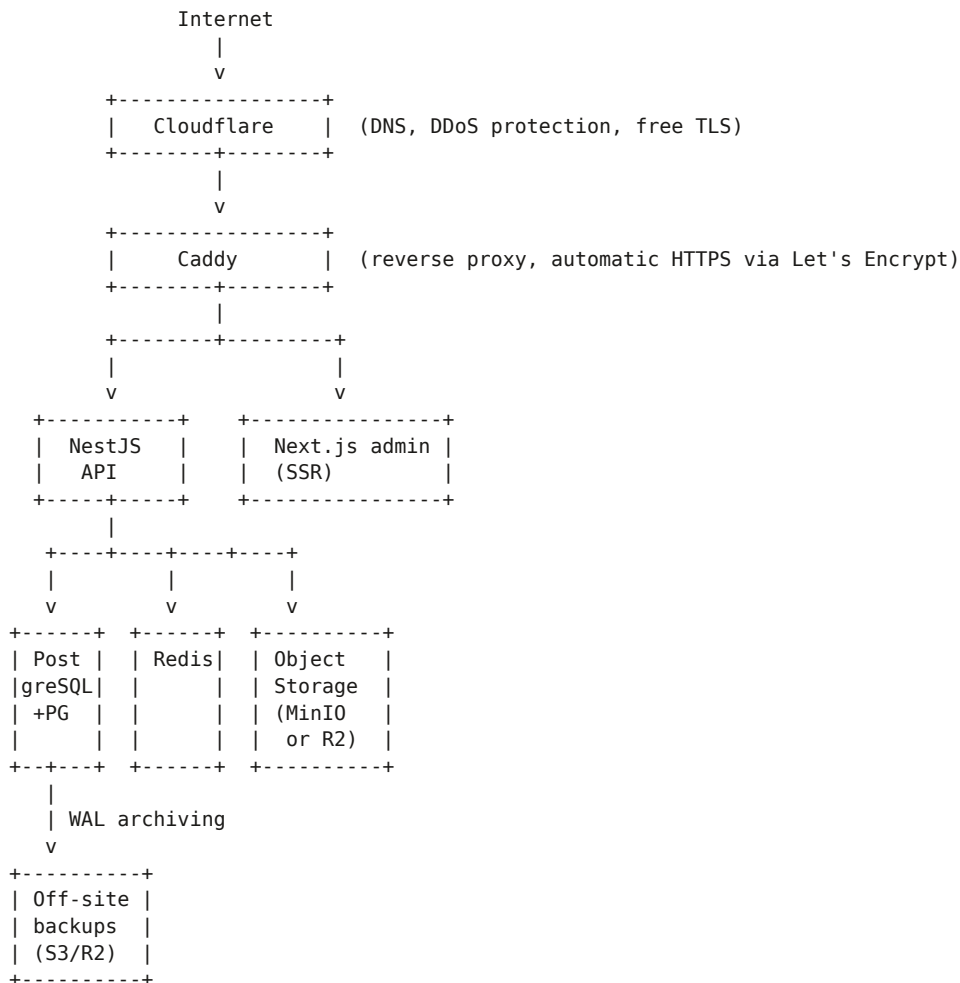


Figure 12.1 — Initial VPS deployment. One machine, one database, one reverse proxy.

12.2 Docker Compose for Local and Prod

The same docker-compose.yml is used in development and production, with environment variables for secrets and a separate docker-compose.prod.yml override for production tweaks (no port exposure to host, read-only filesystems where possible, log drivers, restart policies). This eliminates the "works on my machine" gap between dev and prod.

12.3 Backups: The "Untested Backup Is A Hope" Rule

Having PostgreSQL on the server is not a backup strategy. The platform runs daily full backups (`pg_dump` to compressed SQL) plus continuous WAL archiving via `pgBackRest` or `wal-g`. WAL files stream to off-site object storage (Cloudflare R2 or S3) every few minutes, giving us point-in-time recovery to within 5 minutes of any failure.

Backups are tested by restoring them to a staging database every Sunday at 03:00 EAT. The restore script runs end-to-end without human intervention; if it fails, the on-call engineer gets a critical alert. This is the only way to know the backups actually work — a backup you have never successfully restored is only a hope.

12.4 CI/CD Pipeline

GitHub Actions runs three pipelines: `ci` (on every push: lint, typecheck, unit tests, build), `deploy-staging` (on push to main: build Docker images, push to registry, SSH into staging VPS, `docker compose pull` && `docker compose up -d`), and `deploy-prod` (manual dispatch from a tagged release: same as staging but against prod VPS, with a 5-minute canary window where the new container runs alongside the old one and traffic is shifted over only if error rate stays below threshold).

12.5 Monitoring

Logs are structured JSON emitted to stdout, captured by Docker's `json-file` log driver, shipped to a log aggregator (Loki or plain Elasticsearch for the MVP) and searchable via Grafana or jq. Sentry (self-hosted or cloud) catches uncaught exceptions and frontend errors with full stack traces. UptimeRobot or a similar external probe hits `/health` every minute from outside the VPS and pages the on-call engineer if it fails twice in a row. The principle is: every signal an operator needs to debug an incident should be available without SSHing into the box.

12.6 When to Split Services

The modular monolith becomes a multi-service architecture when — and only when — a specific module develops a scaling or operational need that justifies the cost. Likely candidates, in order of probability:

- **Location ingestion** — if driver count grows past ~500 concurrent, the GPS write path may extract into its own service with its own connection pool and a Kafka-style ingest queue.
- **Payments** — if we add card processing or corporate billing, the payment module may extract to isolate PCI scope.
- **Notifications** — if push/SMS volume grows large enough that the worker pool starves the API, notifications extract into their own queue-consumer service.
- **Admin reporting** — heavy read queries (analytics dashboards) may eventually run against a read replica to avoid contending with transactional writes.

None of these are day-one problems. The modular monolith keeps the seams clean so that when they become problems, the extraction is a refactor — not a rewrite.

13. Architecture Decision Records (ADRs)

Architecture Decision Records capture the "why" behind a decision. Code captures the "what"; ADRs capture the reasoning that led to that "what". Six months from now, when somebody asks "why didn't we use MongoDB?" or "why are we on a single VPS instead of Kubernetes?", the answer lives here — not in someone's memory, not in a deleted Slack thread, not in a guess.

Each ADR uses the standard format: Context, Decision, Consequences, Status. ADRs are numbered and never deleted. If a decision is reversed, the original ADR is marked **Superseded** with a reference to the new ADR that reverses it. This preserves the historical reasoning even after the decision changes.

ADR-001: PostgreSQL + PostGIS as the Source of Truth

Status: Accepted

Context

The platform is a shared, multi-user transactional system with concurrent writers (multiple passengers booking seats on the same cab simultaneously) and a fundamental need for spatial queries (find drivers within X km of a passenger, find pickup points within a service area). SQLite was considered for the centre but is explicitly not recommended for multi-client transactional work — SQLite itself recommends a client/server RDBMS when many clients access the same data and when there are many concurrent writers, because SQLite allows only one writer at a time per database.

Decision

Use PostgreSQL 16+ with the PostGIS extension as the authoritative source of truth for all transactional state. Use SQLite on mobile devices only as an offline cache that syncs through the API.

Consequences

- **Positive:** ACID transactions eliminate double-booking via `SELECT ... FOR UPDATE`. PostGIS gives us spatial indexes and queries natively. Strong ecosystem of tooling (pgBackRest, pgbouncer, pgAdmin).
- **Positive:** SQLite is still part of the architecture, just at the edge — its offline cache role is exactly what SQLite is designed for.
- **Negative:** One more piece of infrastructure to operate (vs. embedded SQLite). Requires a backup strategy, monitoring, and eventual read replicas.
- **Negative:** PostgreSQL must be deployed on a server the team controls — not serverless. This constrains hosting options.

ADR-002: React Native + Expo for Mobile

Status: Accepted

Context

Both passenger and driver apps need to ship on Android (primary Kenyan market) and iOS (smaller but present). The team is already proficient in TypeScript from the web stack. The driver app has hard requirements around GPS, foreground services, and Android 14 background-location permissions.

Decision

Use React Native + Expo + TypeScript for both mobile apps. Flutter was considered and rejected — not because it is bad (it is excellent) but because it would introduce a second language ecosystem (Dart) for the team to maintain, with no compensating benefit for this specific product.

Consequences

- **Positive:** Shared TypeScript types between mobile apps, admin web, and backend. Shared API client package. One mental model.
- **Positive:** Expo Location module supports foreground services, Android 14 background-location permissions, and geofencing.
- **Positive:** Expo Application Services (EAS) handles native builds, OTA updates, and version management without Xcode/Android Studio on every dev machine.
- **Negative:** Native modules required for some platform-specific features may force ejecting from Expo's managed workflow. We accept this risk; the trigger would be a feature Expo cannot support.
- **Negative:** Background location on iOS remains constrained by Apple's 30-second task windows. The driver app is Android-first; iOS driver support is best-effort.

ADR-003: Modular Monolith over Microservices

Status: Accepted

Context

The platform needs to support ~12 bounded contexts (auth, bookings, rides, dispatch, payments, pricing, locations, notifications, admin, audit, plus shared infrastructure). The team is small. The platform has not yet carried its first passenger.

Decision

Build a single NestJS backend with internal module boundaries. Each module owns its own tables, services, and DTOs. Cross-module communication happens via explicit service calls, not via shared database reads. Service extraction is deferred until a specific module develops a specific need.

Consequences

- **Positive:** One deployable, one database, one monitoring dashboard. Dramatically simpler ops for a small team.
- **Positive:** Module boundaries preserve the seams for future service extraction.
- **Positive:** Local development runs with a single docker compose up.
- **Negative:** A bug in one module can crash the whole process. Mitigated by process supervision (Docker restart policies) and by isolating risky operations (e.g., calling M-Pesa) behind retries and circuit breakers.

- **Negative:** Scaling is per-process. If one module needs to scale independently (likely: location ingestion), it must be extracted — but that is a future problem, not a day-one problem.

ADR-004: Adaptive GPS over Continuous Streaming

Status: Accepted

Context

Naive driver tracking would stream one GPS update per second per active driver. For 100 concurrent drivers, that is 100 writes/second to the database — sustainable but wasteful, and it scales linearly with driver count. Worse, it drains driver phone batteries, triggers Android 14 background-service restrictions, and produces historical data of questionable value (you do not need second-by-second resolution to replay a 30-minute trip).

Decision

Driver GPS frequency adapts to operational state: no tracking when off-duty, low frequency (every 5 min) when idle, higher frequency (every 15s) when en route to pickup, active tracking (every 10s) during a trip. Combined with a 50m minimum-distance threshold and battery-level awareness.

Consequences

- **Positive:** Write volume drops ~95% versus naive streaming. Battery life improves. Android background-service limits are respected.
- **Positive:** Historical event table remains queryable for trip replay at 10s resolution — more than enough for dispute resolution.
- **Negative:** "Where is the driver right now?" between pings is interpolated. Acceptable because the driver app sends an immediate ping on state change (e.g., when starting en route).
- **Negative:** Driver app complexity increases — it must track its own state and choose the correct frequency. Mitigated by a single `useLocationTracking` hook that encapsulates the logic.

Appendix A: Pick-up Stations Reference

The platform uses existing SGR waiting/collection points as agreed pickup/drop-off stages. This appendix lists the 25 confirmed stations across the South Coast and North Coast corridors. South Coast stations are carried forward from the initial prototype; North Coast stations are the user-confirmed list. Additional stations may be added by admin via the stations table without code changes — the schema and pricing logic treat them uniformly.

A.1 South Coast Stations

Station Name	Area	Coast	Landmark
Likoni Ferry Container	Likoni	South	Main SGR collection point, Likoni
Fayaz (Kona Mpya)	Likoni	South	Kona Mpya junction
ShikaAdabu (Checkpoint)	Likoni	South	Checkpoint stage
Kombani	Kombani	South	Kombani junction
Naivas Diani	Ukunda	South	Naivas Diani supermarket

A.2 North Coast Stations

Station Name	Area	Coast	Landmark
Kiembeni mwisho	Kiembeni	North	End of Kiembeni road
Kiembeni police	Kiembeni	North	Kiembeni police station
Ananda Marga	Kiembeni	North	Ananda Marga school area
San Sera	Kiembeni	North	San Sera estate
Green Estate	Kiembeni	North	Green Estate residential
Kona Kiembeni	Kiembeni	North	Kiembeni junction
Bamburi Mwisho	Bamburi	North	End of Bamburi road
Naivas	Bamburi	North	Naivas Bamburi supermarket
Total Bamburi	Bamburi	North	Total petrol station, Bamburi
Fisheries	Bamburi	North	Fisheries stage
Mwembeni	Bamburi	North	Mwembeni area
JCC junction	Nyali	North	JCC church junction
Nyali center	Nyali	North	Nyali centre

Station Name	Area	Coast	Landmark
VOK	Nyali	North	VOK stage
Bombolulu	Bombolulu	North	Bombolulu area
Lights	Bombolulu	North	Lights stage

Table A.1 — All 21 confirmed pick-up stations. 5 South Coast + 16 North Coast.

A.3 Train Schedules

Madaraka Express operates six trains daily between Mombasa Terminus (Miritini) and Nairobi Syokimau. The platform treats the three departures from Mombasa and the three arrivals at Mombasa as the operational anchors for both legs of the cab coordination problem.

Time	Direction	Platform Role
08:00	Mombasa → Nairobi	Departure (inbound cab coordination)
15:00	Mombasa → Nairobi	Departure (inbound cab coordination)
22:00	Mombasa → Nairobi	Departure (inbound cab coordination)
04:00	Nairobi → Mombasa	Arrival (outbound cab coordination)
14:00	Nairobi → Mombasa	Arrival (outbound cab coordination)
20:30	Nairobi → Mombasa	Arrival (outbound cab coordination)

A.4 Pricing Rules

Pricing is configurable via the pricing_rules table and may be overridden by admin without code changes. The current configuration is:

Rule	Value	Notes
Base fare (stage pickup, per seat)	KSh 450	Applies to all stations regardless of coast
Off-stage distance surcharge	KSh 50 per km	Charged for off-stage pickup, prorated by distance to nearest station
Off-stage cap	3 km	Beyond 3km the app says "please meet at nearest stage"
Charter multiplier	$1.3 \times (\text{base} \times \text{capacity})$	Private charter pays for all seats plus 30% privacy premium
Nudge discount	10% off	Applied within 60min of cutoff when cab is below 70% fill threshold

Rule	Value	Notes
Peak-hour multiplier	1.0 × (no surcharge yet)	Reserved for future; configurable per pricing_rule row

Table A.2 — Pricing rules as currently configured. All values editable via /admin/pricing.

A.5 Fare Calculation Example

```
-- Pooled, stage pickup, 1 seat
base = 450
surcharge = 0 (stage pickup)
perSeat = 450 + 0 = 450
seats = 1 (pooled)
subtotal = 450 * 1 = 450
charterPremium = 0 (pooled)
TOTAL = 450
```

```
-- Pooled, off-stage 2.4km, 1 seat
base = 450
surcharge = 50 * 2.4 = 120
perSeat = 450 + 120 = 570
seats = 1
subtotal = 570
TOTAL = 570
```

```
-- Charter, stage pickup, 4-seater (whole vehicle)
base = 450
surcharge = 0
perSeat = 450
seats = 4 (whole vehicle)
subtotal = 450 * 4 = 1800
charterPremium = 1800 * 0.30 = 540
TOTAL = 2340
```

```
-- Charter, off-stage 1.5km, 7-seater
base = 450
surcharge = 50 * 1.5 = 75
perSeat = 525
seats = 7
subtotal = 525 * 7 = 3675
charterPremium = 3675 * 0.30 = 1102 (rounded)
TOTAL = 4777
```